Mintermy i maxtermy

- Minterm: funkcja logiczna przyjmująca wartość 1 tylko dla jednej kombinacji wejściowej
- Maxterm: funkcja logiczna przyjmująca wartość 0 tylko dla jednej kombinacji wejściowej

		$\bar{a}\bar{b}\bar{c}$	$\bar{a}b\bar{c}$	$\bar{a}bc$	$a\bar{b}\bar{c}$	$a\bar{b}c$	abc		$a+\bar{b}+c$						$\bar{a}+\bar{b}+\bar{c}$
	abc	$\bar{a}\bar{b}\bar{c}$	$\bar{a}b\bar{c}$	$\bar{a}bc$	$a\bar{b}\bar{c}$	$a\bar{b}c$	abc		$a+\bar{b}+c$						$\bar{a}+\bar{b}+\bar{c}$
0	000	1	0	0	0	0	0	0	0	1	1	1	1	1	1
1	001	0	1	0	0	0	0	0	1	0	1	1	1	1	1
2	010	0	0	1	0	0	0	0	1	1	0	1	1	1	1
3	011	0	0	0	1	0	0	0	1	1	1	0	1	1	1
4	100	0	0	0	0	1	0	0	1	1	1	1	0	1	1
5	101	0	0	0	0	0	1	0	1	1	1	1	1	0	1
6	110	0	0	0	0	0	0	1	1	1	1	1	1	1	0
7	111	0	0	0	0	0	0	0	1	1	1	1	1	1	1

© Jerzy Tyszer 2020

Dekompozycja

$g(x_1x_2x_3 \ldots x_n) = x_1g(1x_2 \ldots x_n) + \bar{x}_1g(0x_2 \ldots x_n)$

$g(x_1x_2x_3 \ldots x_n) = (x_1 + g(0x_2 \ldots x_n))(\bar{x}_1 + g(1x_2 \ldots x_n))$

Dowód przez podstawienie 0 i 1 w miejsce x_1

Twierdzenie dla 2 zmiennych:

$$\begin{aligned} g(x_1x_2x_3 \ldots x_n) = {} & x_1x_2g(11x_3 \ldots x_n) + \\ & + x_1\bar{x}_2g(10x_3 \ldots x_n) + \\ & + \bar{x}_1x_2g(01x_3 \ldots x_n) + \\ & + \bar{x}_1\bar{x}_2g(00x_3 \ldots X_n) \end{aligned}$$

Twierdzenie błędnie przypisywane C. Shannon'owi

© Jerzy Tyszer 2020

Pełna postać kanoniczna dysjunkcyjna

Każdą funkcję logiczną można zapisać jednoznacznie jako sumę mintermów

$$g(x_1x_2x_3 \ldots x_n) = \sum_{k=0}^{2^n-1} g_k m_k$$

g_k – wartość funkcji dla k-tej kombinacji wejściowej

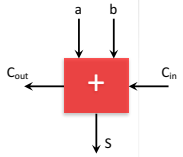
m_k – minterm z wartością 1 dla k-tej kombinacji wejściowej

Notacja: $g(x_1x_2x_3 \ldots x_n) = \Sigma m(p_1, p_2, \ldots, p_s)$

© Jerzy Tyszer 2020

Sumator 1-bitowy

	a	b	C_{in}	S	C_{out}
0	0	0	0	0	0
1	0	0	1	1	0
2	0	1	0	1	0
3	0	1	1	0	1
4	1	0	0	1	0
5	1	0	1	0	1
6	1	1	0	0	1
7	1	1	1	1	1



$S = \Sigma m\ (1, 2, 4, 7)$

$C_{out} = \Sigma m\ (3, 5, 6, 7)$

$S = \bar{a}\bar{b}c_{in} + \bar{a}b\bar{c}_{in} + a\bar{b}\bar{c}_{in} + abc_{in}$

$C_{out} = \bar{a}bc_{in} + a\bar{b}c_{in} + ab\bar{c}_{in} + abc_{in}$

© Jerzy Tyszer 2020

Pełna postać kanoniczna koniunkcyjna

Każdą funkcję logiczną można zapisać jednoznacznie jako iloczyn maxtermów

$$g(x_1x_2x_3 \ldots x_n) = \prod_{k=0}^{2^n-1} (g_k + M_k)$$

g_k – wartość funkcji dla k-tej kombinacji wejściowej

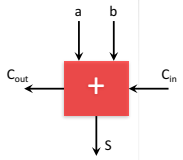
M_k – maxterm z wartością 0 dla k-tej kombinacji wejściowej

Notacja: $g(x_1x_2x_3 \ldots x_n) = \Pi M(p_1, p_2, \ldots, p_s)$

© Jerzy Tyszer 2020

Sumator 1-bitowy

	a	b	C_{in}	S	C_{out}
0	0	0	0	0	0
1	0	0	1	1	0
2	0	1	0	1	0
3	0	1	1	0	1
4	1	0	0	1	0
5	1	0	1	0	1
6	1	1	0	0	1
7	1	1	1	1	1



$S = \Pi M\ (0, 3, 5, 6)$

$C_{out} = \Pi M\ (0, 1, 2, 4)$

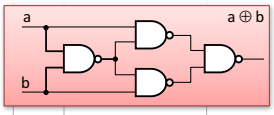
$S = (a+b+c_{in})\ (a+\bar{b}+\bar{c}_{in})\ (\bar{a}+b+\bar{c}_{in})\ (\bar{a}+\bar{b}+c_{in})$

$C_{out} = (a+b+c_{in})\ (a+b+\bar{c}_{in})\ (a+\bar{b}+c_{in})\ (\bar{a}+b+c_{in})$

© Jerzy Tyszer 2020

Suma modulo 2

- Równa 1 dla nieparzystej liczby argumentów = 1
- Podstawowe zależności
  - $a \oplus b = \bar{a}b + a\bar{b} = (a + b)(\bar{a} + \bar{b})$
  - $\overline{a \oplus b} = a \oplus \bar{b} = \bar{a} \oplus b = ab + \bar{a}\bar{b} = (\bar{a} + b)(a + \bar{b})$
  - $a \oplus a = 0 \quad a \oplus \bar{a} = 1$
  - $a \oplus 1 = \bar{a} \quad a \oplus 0 = a$ sterowany negator
  - $a (b \oplus c) = ab \oplus ac$
  - $a + b = a \oplus b \oplus ab$
  - $a \oplus (a + b) = \bar{a}b$
  - $a \oplus ab = a\bar{b}$

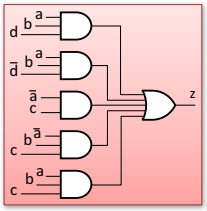


© Jerzy Tyszer 2020

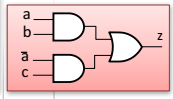
Rola minimalizacji

- Uproszczenie realizacji jako wynik
  - redukcji liczby składników (czynników) – bramki
  - redukcji liczby literałów – liczba wejść bramek
- Przykład: $z = abd + ab\bar{d} + \bar{a}c + \bar{a}bc + abc$

5 składników, 14 literałów



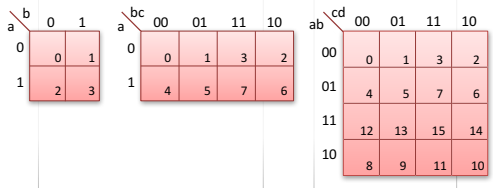
$z = ab + \bar{a}c$
2 składniki, 4 literały



© Jerzy Tyszer 2020

Siatki Karnaugh

Siatka Karnaugh jest uporządkowaną strukturą prostokątną złożoną z 2^n kratek, z których każda reprezentuje jeden minterm; dwie dowolne sąsiednie kratki różni wartość tylko jednej zmiennej

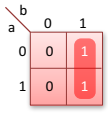


© Jerzy Tyszer 2020

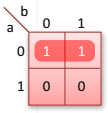
Siatki Karnaugh

Podstawowa reguła: twierdzenie o minimalizacji

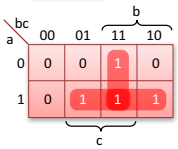
$a\,b + a\,\bar{b} = a\,(b + \bar{b}) = a$



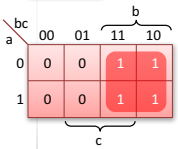
$g(a, b) = b$



$g(a, b) = \bar{a}$



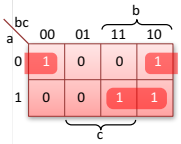
$g(a, b, c) = ab + bc + ac$



$g(a, b, c) = b$

© Jerzy Tyszer 2020

Dalsze przykłady



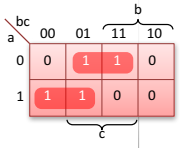
$g(a, b, c) = \Sigma m(0, 2, 6, 7)$

$g(a, b, c) = \bar{a}\bar{c} + ab$

Sąsiedztwo kratek dotyczy także skrajnych kolumn i skrajnych wierszy

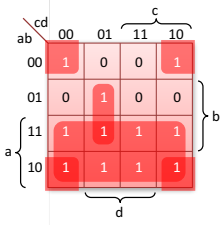
$\overline{g(a, b, c)} = \Sigma m(1, 3, 4, 5)$

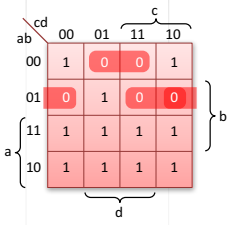
$\overline{g(a, b, c)} = a\bar{b} + \bar{a}c$



© Jerzy Tyszer 2020

Przykłady – 4 zmienne





$g(a, b, c, d) = \Sigma m(0, 2, 5, 8, 9, 10, 11, 12, 13, 14, 15)$

$g(a, b, c, d) = a + b\bar{c}d + \bar{b}\bar{d}$ ⟵ dla jedynek

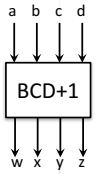
dla zer ⟶ $g(a, b, c, d) = (a + b + \bar{d})(a + \bar{b} + d)(a + \bar{b} + \bar{c})$

© Jerzy Tyszer 2020

Funkcje niezupełne

- Dla pewnych kombinacji wejściowych wartość funkcji jest nieokreślona
- Oznaczenie: x lub DC (*Don't care*)
- Przykład: układ dodający 1 do liczby BCD

a b c d	w x y z	a b c d	w x y z
0 0 0 0	0 0 0 1	1 0 0 0	1 0 0 1
0 0 0 1	0 0 1 0	1 0 0 1	0 0 0 0
0 0 1 0	0 0 1 1	1 0 1 0	x x x x
0 0 1 1	0 1 0 0	1 0 1 1	x x x x
0 1 0 0	0 1 0 1	1 1 0 0	x x x x
0 1 0 1	0 1 1 0	1 1 0 1	x x x x
0 1 1 0	0 1 1 1	1 1 1 0	x x x x
0 1 1 1	1 0 0 0	1 1 1 1	x x x x

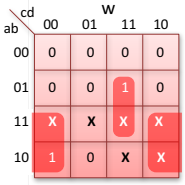


$z = \Sigma m(0, 2, 4, 6, 8) + d(10, 11, 12, 13, 14, 15)$

© Jerzy Tyszer 2020

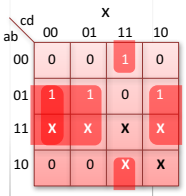
Przykład: BCD + 1

Wartości 0 lub 1 zastępują symbol x, jeżeli redukuje to literały



$w(a, b, c, d) = bcd + a\bar{d}$

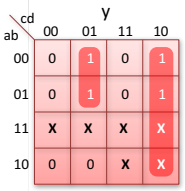
$= \bar{a}bcd + a\bar{b}\bar{c}\bar{d}$ bez użycia x



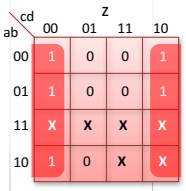
$x(a, b, c, d) = b\bar{c} + b\bar{d} + \bar{b}cd$

© Jerzy Tyszer 2020

Przykład: BCD + 1

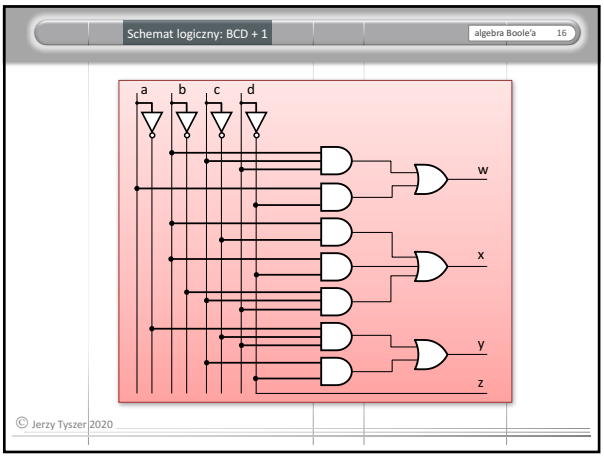


$y(a, b, c, d) = \bar{a}\bar{c}d + c\bar{d}$



$z(a, b, c, d) = \bar{d}$

© Jerzy Tyszer 2020


Schemat logiczny: BCD + 1
algebra Boole'a 16
a
b
c
d
w
x
y
z
© Jerzy Tyszer 2020

Nowe terminy

- Implikant: minterm lub mintermy połączone w siatce Karnaugh
- Implikant prosty: implikant, którego nie można rozszerzyć celem wyeliminowania zmiennej
- Implikant istotny: implikant prosty, który zawiera minterm nie występujący w żadnym innym implikancie prostym
- W podobny sposób definiujemy implicenty

© Jerzy Tyszer 2020

Reguły minimalizacji

- Rozszerzaj implikanty do implikantów prostych
- Utwórz pokrycie funkcji logicznej za pomocą minimalnej liczby implikantów prostych
- Implikanty istotne są bezwarunkowo elementami pokrycia funkcji

ab \ cd	00	01	11	10
00	0	1	1	0
01	1	1	1	0
11	1	0	1	1
10	0	0	1	1

6 implikantów prostych:
$b\bar{c}\bar{d}$, $\bar{a}b\bar{c}$, $\bar{a}d$, cd, ac, $ab\bar{d}$
Implikanty istotne:
$\bar{a}d$, ac
Minimalne pokrycie:
$\bar{a}d + ac + b\bar{c}\bar{d}$

© Jerzy Tyszer 2020

Dalsze przykłady

ab \ cd	00	01	11	10
00	0	0	1	0
01	1	1	1	0
11	0	1	1	1
10	0	1	0	0

5 implikantów prostych:
bd, $\bar{a}b\bar{c}$, $\bar{a}cd$, abc, $ac\bar{d}$
Implikanty istotne:
$\bar{a}b\bar{c}$, $\bar{a}cd$, abc, $ac\bar{d}$
Minimalne pokrycie
tworzą implikanty istotne

4 implikanty proste:
bd, ab, ac, $a\bar{d}$
Implikanty istotne:
bd, ac, $a\bar{d}$
Minimalne pokrycie
tworzą implikanty istotne

ab \ cd	00	01	11	10
00	0	0	0	0
01	0	1	1	0
11	1	1	1	1
10	1	0	1	1

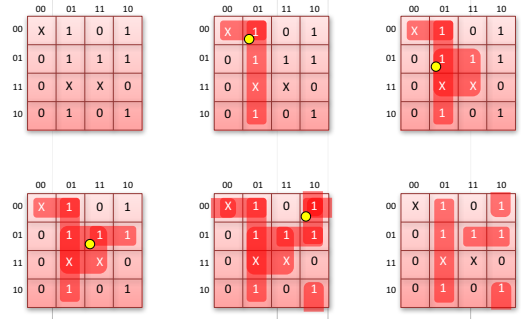
© Jerzy Tyszer 2020

Algorytm minimalizacji

- Dla każdego mintermu *m* znajdź wszystkie implikanty proste zawierające *m* uwzględniając wartości x; pamiętaj, że implikant składa się z liczby elementów będącej potęgą 2
- Wybierz wszystkie implikanty istotne
- Znajdź minimalne pokrycie (implikanty proste) dla tych mintermów, które nie należą do pokrycia utworzonego przez implikanty istotne

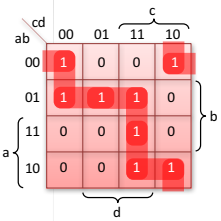
© Jerzy Tyszer 2020

Przykład



© Jerzy Tyszer 2020

Metoda Petricka



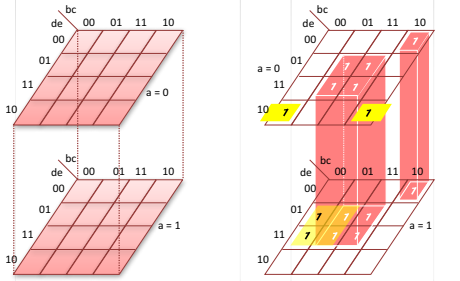
Brak implikantów istotnych!
Który zestaw implikantów prostych jest wystarczający?

$$P_{x,y} = \begin{cases} 1, \text{ jeżeli użyjemy implikantu} \\ \quad \text{z kwadratów x oraz y} \\ 0, \text{ w przeciwnym razie} \end{cases}$$

$$(P_{0,4} + P_{0,2})(P_{0,4} + P_{4,5})(P_{4,5} + P_{5,7})(P_{5,7} + P_{7,15}) \times$$
$$\times (P_{7,15} + P_{11,15})(P_{11,15} + P_{10,11})(P_{10,11} + P_{2,10})(P_{2,10} + P_{0,2}) =$$
$$= (P_{0,4} + P_{0,2}P_{4,5})(P_{5,7} + P_{4,5}P_{7,15})(P_{11,15} + P_{7,15}P_{10,11}) \times$$
$$\times (P_{2,10} + P_{10,11}P_{0,2}) = P_{0,4}P_{5,7}P_{11,15}P_{2,10} + P_{0,2}P_{4,5}P_{7,15}P_{10,11} + \ldots$$

© Jerzy Tyszer 2020

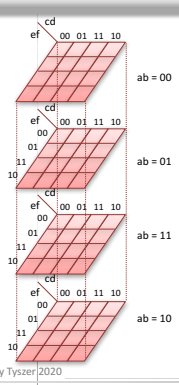
Karnaugh – 5 zmiennych



$$g(a, b, c, d, e) = ce + a\bar{b}e + b\bar{c}\bar{d}\bar{e} + \bar{a}\bar{c}d\bar{e}$$

© Jerzy Tyszer 2020

Karnaugh – 6 zmiennych



Funkcje logiczne z dużą liczbą zmiennych wymagają bardziej efektywnych metod reprezentacji i przetwarzania



© Jerzy Tyszer 2020

M. Quine'a-McCluskeya

$$g(a, b, c, d) = \sum m(4, 5, 6, 8, 9, 10, 13) + d(0, 7, 15)$$

(1)		(2)			(3)	
0	0000 √	0, 4	0-00	*	4, 5, 6, 7	01-- *
		0, 8	-000	*	5, 7, 13, 15	-1-1 *
4	0100 √					
8	1000 √	4, 5	010-	√		
		4, 6	01-0	√		
5	0101 √	8, 9	100-	*		
6	0110 √	8, 10	10-0	*		
9	1001 √					
10	1010 √	5, 7	01-1	√		
		5, 13	-101	√		
7	0111 √	6, 7	011-	√		
13	1101 √	9, 13	1-01	*		
15	1111 √	7, 15	-111	√		
		13, 15	11-1	√		

Implikanty proste

0-00 = a'c'd'
-000 = b'c'd'
100- = ab'c'
10-0 = ab'd'
1-01 = ac'd
01-- = a'b
-1-1 = bd

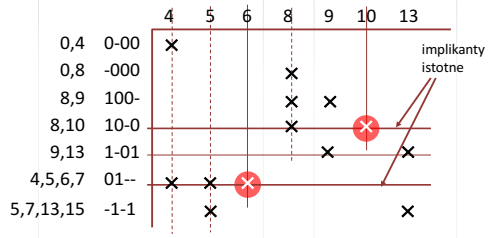
© Jerzy Tyszer 2020

M. Quine'a-McCluskeya

- Systematyczna generacja wszystkich implikantów prostych
- Podstawowe kroki
  - pogrupuj wszystkie mintermy wg liczby jedynek w ich reprezentacjach binarnych
  - porównaj każdy minterm z jednej grupy z każdym mintermem z następnej grupy; jeżeli różnią się jedną pozycją, sklej ze sobą, eliminując jedną ze zmiennych – wynik zapisz w następnej kolumnie
  - wykonaj poprzedni krok dla wszystkich grup i dla wszystkich tworzonych kolejno kolumn
  - wybierz minimalny zbiór implikantów prostych

© Jerzy Tyszer 2020

Implikanty proste

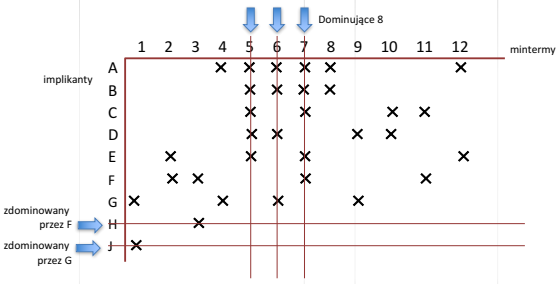


$g(a, b, c, d) = a\bar{b}\bar{d} + \bar{a}b + \bar{a}\bar{c}d$

Na tym etapie mintermy reprezentujące wartości x nie są brane pod uwagę

© Jerzy Tyszer 2020

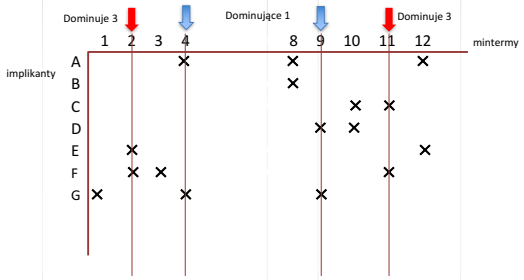
Dominacja



Wykreślamy dominujące kolumny
Wykreślamy zdominowane wiersze

© Jerzy Tyszer 2020

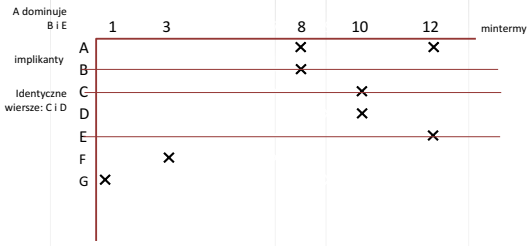
Dominacja



Wykreślamy dominujące kolumny
Wykreślamy zdominowane wiersze

© Jerzy Tyszer 2020

Dominacja

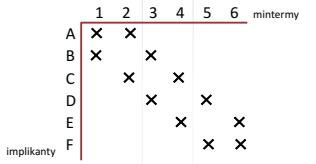


Z identycznych wierszy pozostawiamy jeden

Rozwiązanie = A + D + F + G

© Jerzy Tyszer 2020

Funkcja pokrycia



$\Phi(A, B, C, D, E, F) = (A + B)(A + C)(B + D)(C + E)(D + F)(E + F) =$
$= ADE + ACDF + ABEF + BCDE + BCF$

Alternatywnie, wybierz jeden implikant i powtórz procedurę dominacji.
Arbitralne wybory nie gwarantują rozwiązania optymalnego –
konieczność sprawdzenia wielu wariantów.

© Jerzy Tyszer 2020

Espresso

- Notacja w postaci kostek (*cubes*)
- Kostka reprezentuje mintermy lub wynik ich sklejeń
- Przykład: funkcja przeniesienie C_{out}

$$\left\{\begin{matrix}0\,1\,1\\1\,0\,1\\1\,1\,0\\1\,1\,1\end{matrix}\right\} \Rightarrow \left\{\begin{matrix}x\,1\,1\\1\,x\,1\\1\,1\,x\end{matrix}\right\}$$

1. Rozszerz kostki do postaci implikantów prostych
2. Znajdź lokalnie optymalne pokrycie funkcji
3. Zredukuj wybrane kostki, zachowując pełne pokrycie
4. Powtarzaj kroki 1–3, dopóki nowe pokrycia funkcji są lepsze od znalezionych w poprzednich iteracjach

© Jerzy Tyszer 2020

Zastosowanie Espresso


```
.i 4
.o 1
.ilb a b c d
.ob g
.p 12
0000 1
0100 1
0001 1
0101 1
1101 1
1001 1
0010 1
0110 1
1110 1
1010 1
1111 1
1011 1
.e
```

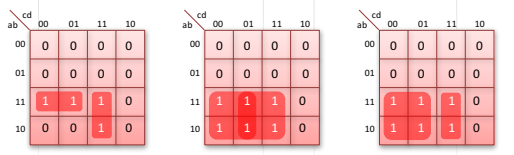
kierunek ekspansji i redukcji do wyboru

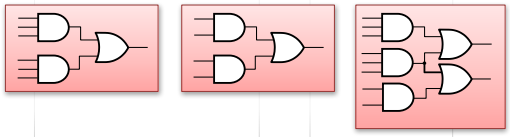
Ekspansja	Redukcja	Ekspansja	Pokrycie	
0x0x	0x0x	0x0x	0x0x	1
xx01	1x01	1xx1	1xx1	1
xx10	0x10	xx10	xx10	1
1x1x	1x1x	1x1x		

w każdym kroku pełne rozwinięcie kostek musi odpowiadać liście kostek wyjściowych

© Jerzy Tyszer 2020

Funkcje wielowyjściowe



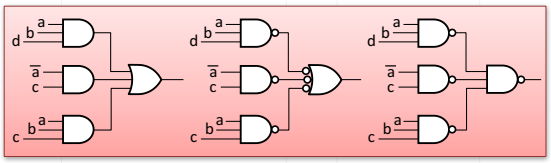


Problem znajdowania wspólnych implikantów

© Jerzy Tyszer 2020

Układy NAND–NAND

Układy typu OR–AND można zamienić na równoważne struktury 2-poziomowe zrealizowane z bramek NAND



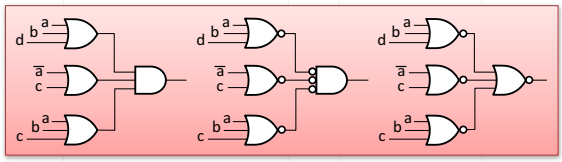
Bezpośrednie zastosowanie praw de Morgana

Podobna procedura dla układów o dowolnej strukturze

© Jerzy Tyszer 2020

Układy NOR–NOR

Układy typu AND–OR można zamienić na równoważne struktury 2-poziomowe zrealizowane z bramek NOR



Bezpośrednie zastosowanie praw de Morgana

Podobna procedura dla układów o dowolnej strukturze

© Jerzy Tyszer 2020

Układy wielopoziomowe

- Transformacje układów dwupoziomowych pozwalają na dalszą redukcję bramek logicznych i liczby ich wejść
- Podstawowe operacje
  - dekompozycja (*decomposition*)
  - faktoryzacja (*factoring*)
- Transformacje zwiększają liczbę poziomów w układzie (opóźnienie!), zmniejszając jednak koszt realizacji układu

© Jerzy Tyszer 2020

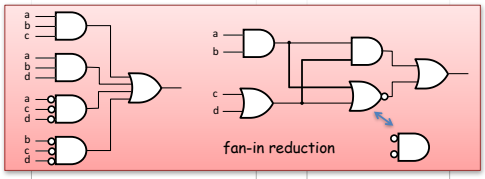
Dekompozycja

Zastępuje pojedyncze wyrażenie zbiorem nowych wyrażeń pomocniczych

$$F = abc + abd + \bar{a}\bar{c}\bar{d} + \bar{b}\bar{c}\bar{d}$$

może być zapisane jako

$$F = \bar{X}\bar{Y} + XY \qquad X = ab \quad Y = c + d$$

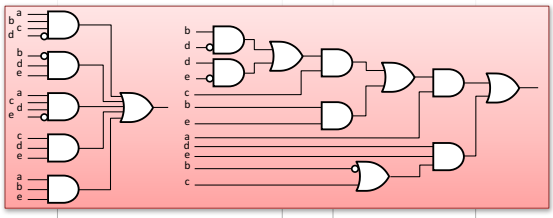


© Jerzy Tyszer 2020

Faktoryzacja

Wyraża sumę iloczynów (lub iloczyn sum) w postaci wielopoziomowej z minimalną liczbą literałów

$h(a, b, c, d, e) = abcd' + b'de + acde' + cde + abe$
$= a(c(bd' + de') + be) + de(b' + c)$



© Jerzy Tyszer 2020

Kody

- Zakodowanie 2^n liczb wymaga minimum n bitów
- Kody dwójkowo-dziesiętne BCD (*binary coded decimal*)
- Alfanumeryczne (ASCII, EBCDIC)
- Refleksyjne (Graya)
- Wykrywające błędy
  - bit(y) parzystości lub bit(y) nieparzystości
  - stałowagowe „m z n": 1 z 10 (pierścieniowy), 2 z 5
  - resztowe: do liczby N dołącz resztę z dzielenia N przez podstawę m

© Jerzy Tyszer 2020

Kody BCD

- Liczba kodów BCD
  - 16 kombinacji użytych do zakodowania 10 cyfr
  - łącznie ponad 2,9 $\cdot 10^{10}$ kodów $\rightarrow$ 16! / 6!
  - tylko niektóre mają znaczenie praktyczne
- Kod BCD 8421
  - 186 = 0001 1000 0110
  - porównaj: 186 = 10111010 (binarny kod pozycyjny)
- Kody samouzupełniające
  - uzupełnienie do 1 liczb dwójkowych daje uzupełnienie do 9 odpowiednich liczb dziesiętnych
  - przykłady: 2421 (Aikena), 84 –2 –1, XS3 (*Excess*-3)

© Jerzy Tyszer 2020

Kody – przykłady

	XS3	2421	2 z 5
0	0011	0000	00011
1	0100	0001	00101
2	0101	0010	01001
3	0110	0011	10001
4	0111	0100	00110
5	1000	1011	01010
6	1001	1100	10010
7	1010	1101	01100
8	1011	1110	10100
9	1100	1111	11000

Graya

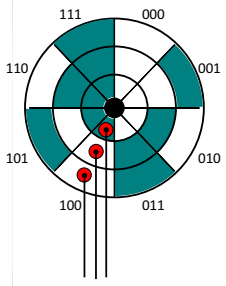
0	00	000
1	01	001
	11	011
	10	010
		110
		111
		101
		100



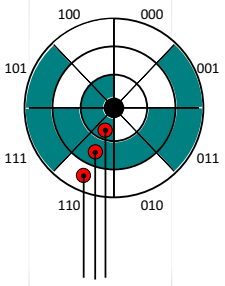
$G_i = b_{i+1} \oplus b_i$

© Jerzy Tyszer 2020

Optyczny wałek kodujący

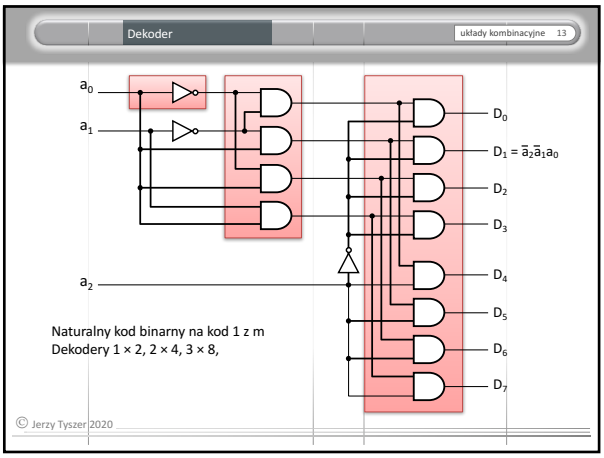


000, 111, 001, 010, 011, 100, 101, 110



000, 100

© Jerzy Tyszer 2020


Dekoder
układy kombinacyjne 13
a0
a1
a2
D0
D1 = ā2ā1a0
D2
D3
D4
D5
D6
D7
Naturalny kod binarny na kod 1 z m
Dekodery 1 × 2, 2 × 4, 3 × 8,
© Jerzy Tyszer 2020

Kody

- Zakodowanie 2^n liczb wymaga minimum n bitów
- Kody dwójkowo-dziesiętne BCD (*binary coded decimal*)
- Alfanumeryczne (ASCII, EBCDIC)
- Refleksyjne (Graya)
- Wykrywające błędy
  - bit(y) parzystości lub bit(y) nieparzystości
  - stałowagowe „m z n”: 1 z 10 (pierścieniowy), 2 z 5
  - resztowe: do liczby N dołącz resztę z dzielenia N przez podstawę m

© Jerzy Tyszer 2017

Kody BCD

- Liczba kodów BCD
  - 16 kombinacji użytych do zakodowania 10 cyfr
  - łącznie ponad $2{,}9 \cdot 10^{10}$ kodów $\rightarrow$ 16! / 6!
  - tylko niektóre mają znaczenie praktyczne
- Kod BCD 8421
  - 186 = 0001 1000 0110
  - porównaj: 186 = 10111010 (binarny kod pozycyjny)
- Kody samouzupełniające
  - uzupełnienie do 1 liczb dwójkowych daje uzupełnienie do 9 odpowiednich liczb dziesiętnych
  - przykłady: 2421 (Aikena), 84 –2 –1, XS3 (*Excess*-3)

© Jerzy Tyszer 2017

Kody – przykłady

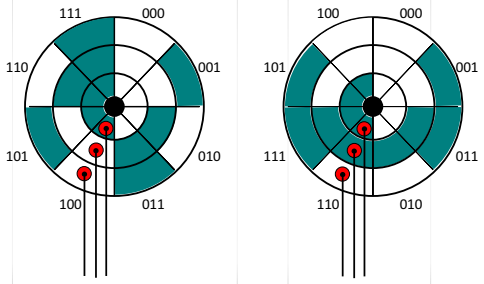
	XS3	2421	2 z 5
0	0011	0000	00011
1	0100	0001	00101
2	0101	0010	01001
3	0110	0011	10001
4	0111	0100	00110
5	1000	1011	01010
6	1001	1100	10010
7	1010	1101	01100
8	1011	1110	10100
9	1100	1111	11000

	Graya	
0	00	000
1	01	001
	11	011
	10	010
		110
		111
		101
		100

$G_i = b_{i+1} \oplus b_i$

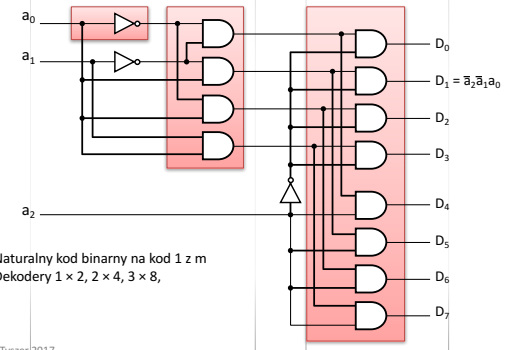
© Jerzy Tyszer 2017

Optyczny wałek kodujący



© Jerzy Tyszer 2017

Dekoder



Naturalny kod binarny na kod 1 z m
Dekodery 1 × 2, 2 × 4, 3 × 8,

© Jerzy Tyszer 2017

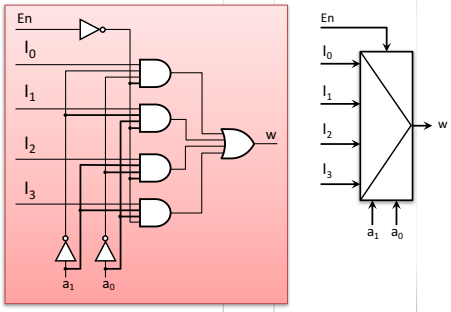
Multipleksery

- Umożliwiają wybór i przesłanie na wyjście sygnału z jednego z N wejść informacyjnych
- Wejście jest wybierane za pomocą wejść adresowych (sterujących)
- Dodatkowe wejście *enable* pozwala zablokować stan wyjścia multipleksera
- Typowo liczba wejść informacyjnych jest związana z liczbą wejść sterujących zależnością

$$N = 2^n$$

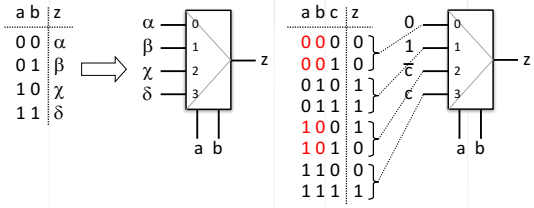
© Jerzy Tyszer 2017

Multipleksery



© Jerzy Tyszer 2017

Multipleksery i funkcje



Dla większej liczby zmiennych wejściowych konieczna rozbudowa układu od strony wejść multipleksera

© Jerzy Tyszer 2017

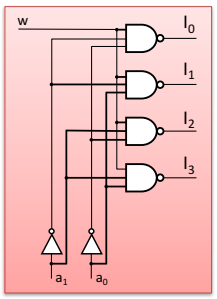
Demultipleksery

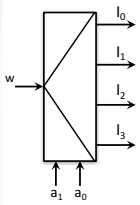
- Umożliwiają wybór i przesłanie sygnału z wejścia do jednego z N wyjść
- Wyjście jest wybierane za pomocą wejść adresowych (sterujących)
- Stosowane jako dekodery z naturalnego kodu binarnego na kod „1 z N”
- Typowo liczba wyjść informacyjnych jest związana z liczbą wejść sterujących zależnością

$$N = 2^n$$

© Jerzy Tyszer 2017

Demultipleksery

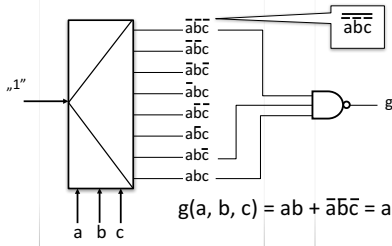




© Jerzy Tyszer 2017

Funkcje wielowyjściowe

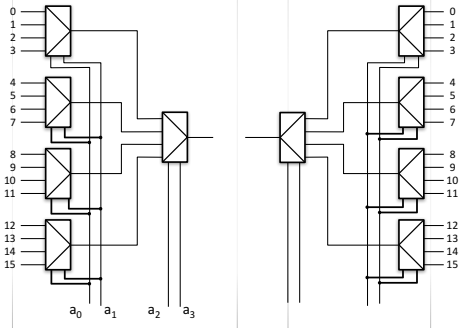
Na wyjściach demultipleksera są dostępne realizacje wszystkich mintermów n zmiennych



$g(a, b, c) = ab + \bar{a}\bar{b}\bar{c} = abc + ab\bar{c} + \bar{a}\bar{b}\bar{c}$

© Jerzy Tyszer 2017

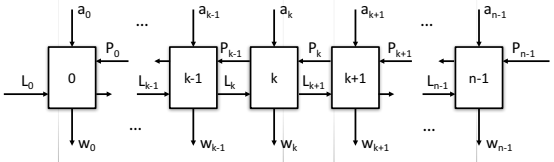
Rozbudowa



© Jerzy Tyszer 2017

Układy iteracyjne

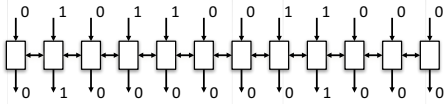
- Układy wielowejściowe zbudowane z bloków o podobnej strukturze
- Jednokierunkowy lub dwukierunkowy przepływ informacji
- Problem szybkości działania



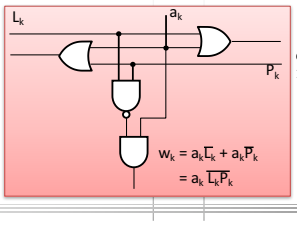
© Jerzy Tyszer 2017

Układy iteracyjne

Dwie skrajne jedynki z kombinacji wejściowej są przesyłane na odpowiadające im wyjścia; pozostałe wyjścia – 0



co najmniej jedna 1 z lewej strony

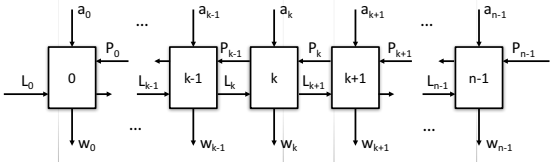


co najmniej jedna 1 z prawej strony

© Jerzy Tyszer 2017

Układy iteracyjne

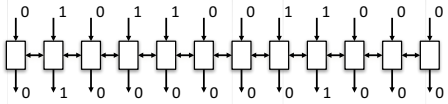
- Układy wielowejściowe zbudowane z bloków o podobnej strukturze
- Jednokierunkowy lub dwukierunkowy przepływ informacji
- Problem szybkości działania



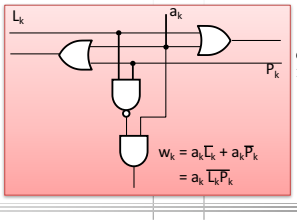
© Jerzy Tyszer 2017

Układy iteracyjne

Dwie skrajne jedynki z kombinacji wejściowej są przesyłane na odpowiadające im wyjścia; pozostałe wyjścia – 0



co najmniej jedna 1 z lewej strony

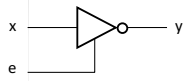


co najmniej jedna 1 z prawej strony

© Jerzy Tyszer 2017

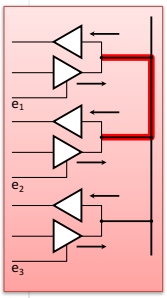
Bramki trójstanowe TTL

Inverter



x	e	y
0	0	1
0	1	Z
1	0	0
1	1	Z

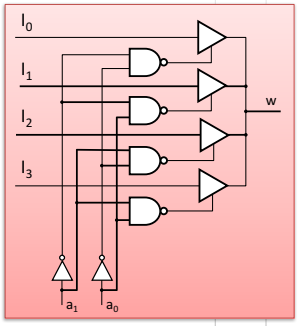
Stan wysokiej impedancji



podłączenie do szyny

© Jerzy Tyszer 2017

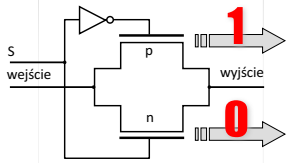
Zastosowania



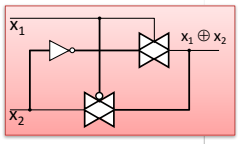
multiplekser

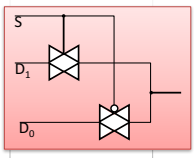
© Jerzy Tyszer 2017

Bramki transmisyjne



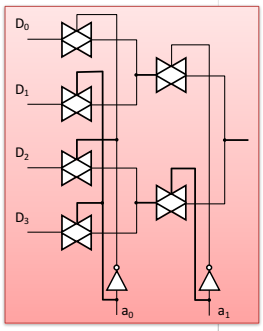
we	S	wy
0	0	Z
1	0	Z
0	1	0
1	1	1





© Jerzy Tyszer 2017

Dalsze przykłady



mux 4×1

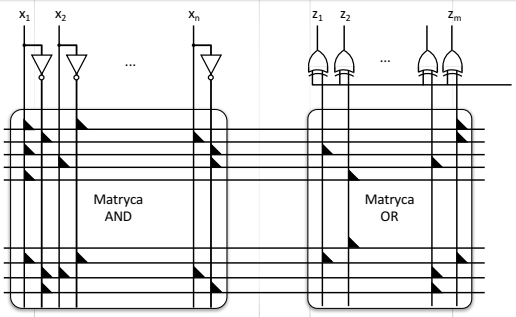
© Jerzy Tyszer 2017

Układy programowalne

- Standardowe układy indywidualnie konfigurowane
- Programowalne struktury logiczne (PLD – *programmable logic devices*)
  - PROM – programowalna pamięć nieulotna – najprostszy (choć nieefektywny) sposób realizacji funkcji logicznej
  - (F)PLA – (*field*) *programmable logic arrays* – bardziej elastyczny sposób formowania funkcji logicznych
  - PAL/GAL – *programmable / generic array logic*
- Programowalne matryce bramkowe (FPGA – *field programmable gate arrays*)

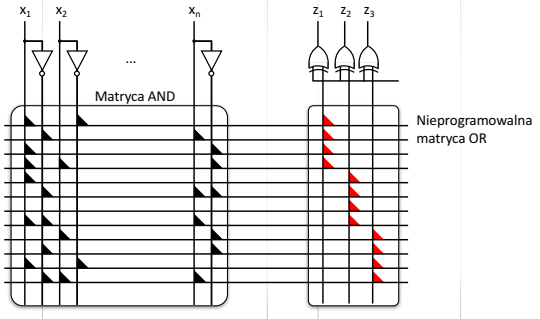
© Jerzy Tyszer 2017

Układ FPLA



© Jerzy Tyszer 2017

Układy PAL/GAL



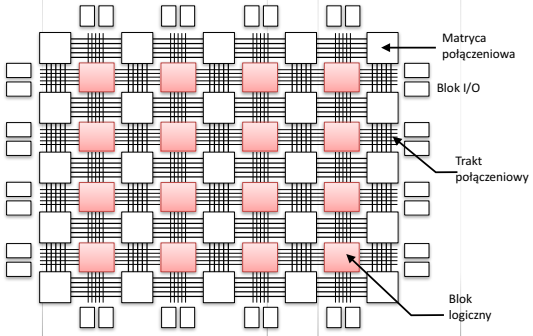
© Jerzy Tyszer 2017

Układy FPGA

- Bloki logiczne rozmieszczone w strukturze matrycy i otoczone kanałami łączeniowymi
- Programowanie – wykonanie połączeń wewnątrz bloków oraz między nimi
- Bloki zawierają małe pamięci realizujące wymagane funkcje logiczne (*look-up tables*)
- Możliwość szybkiej budowy prototypów
- Projektowanie w pełni zautomatyzowane (ViewLogic, Synopsys, Cadence, Mentor Graphics)

© Jerzy Tyszer 2017

Układ FPGA



© Jerzy Tyszer 2017

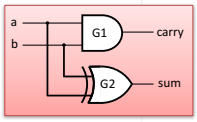
Języki HDL

- Języki opisu sprzętu (HDL – *hardware description languages*)
- Opis w postaci behawioralnej lub strukturalnej do przetwarzania automatycznego
- CUPL (Logical Devices), ABEL (Data I/O), AHDL (Altera), PLDesigner
- VHDL (Departament Obrony USA - 1981, standard IEEE - 1987, norma Unii Europejskiej)
- Verilog (oparty na C, Gateway Design Automation – 1985, obecnie Cadence, standard IEEE - 1995)

© Jerzy Tyszer 2017

Przykład w języku VHDL


```
architecture STRUCTURE of HALF_ADDER is
      component AND2
            port (x, y: in bit; o: out bit);
      component EXOR2
            port (x, y: in bit; o: out bit);
      begin
            G1: AND2
            port map (a, b, carry);
            G2: EXOR2
            port map  (a, b, sum);
      end STRUCTURE;
```




```
                        architecture BEHAVIOR of HALF_ADDER is
                        process
                              begin
                                    carry <= (a and b);
                                    sum <= (a xor b);
                               end process;
                        end BEHAVIOR;

                        entity HALF_ADDER is
                              port (a, b: in bit; sum, carry: out bit);
                        end HALF_ADDER
```

© Jerzy Tyszer 2017

Układy ASIC

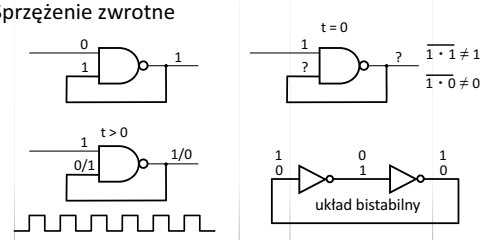
- ASIC – *application specific integrated circuits*
  - matryce bramkowe (GA – *gate array*)
  - matryce komórkowe (SC – *standard cell libraries*)
  - układy indywidualne (FC – *full custom*)
- Układy VLSI wytwarzane do ściśle określonych zastosowań, z reguły przy przewidywanej skali produkcji nie mniejszej niż 10 000 sztuk
- Prof. Jan Czochralski (1885 – 1953), twórca metody otrzymywania monokryształów (1916) – nadal stosowana na całym świecie



© Jerzy Tyszer 2017

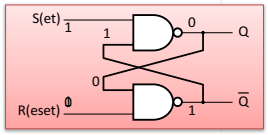
Prolog

- Układy kombinacyjne – jednokierunkowy przepływ sygnału (*feed-forward circuits*) określony warstwami układu
- Sprzężenie zwrotne

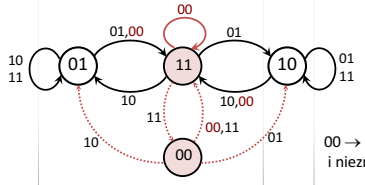


© Jerzy Tyszer 2017

Zatrzask RS



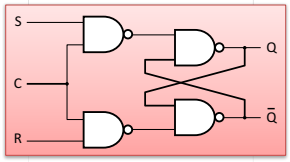
S	R	Q	$\overline{Q}$
1	0	0	1
1	1	0	1
0	1	1	0
1	1	1	0
0	0	1	1



$00 \rightarrow 11$ możliwe oscylacje i nieznany stan końcowy

© Jerzy Tyszer 2017

Zatrzaski bramkowane

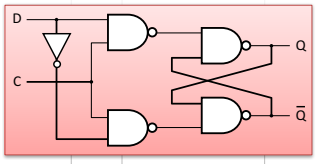


S	R	Q_t	Q_{t+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1

zatrzask typu D

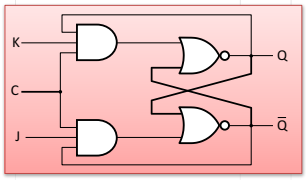
przepisz dane na wyjście, gdy C = 1

stan SR = 11 nie wystąpi



© Jerzy Tyszer 2017

Zatrzask typu JK



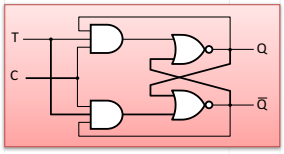
J	K	Q_t	Q_{t+1}	
0	0	0	0	nop
0	0	1	1	
0	1	0	0	reset
0	1	1	0	
1	0	0	1	set
1	0	1	1	
1	1	0	1	toggle
1	1	1	0	

Dlaczego JK:
Jack Kilby (twórca pierwszego układu scalonego)
John Kennedy (prezydent USA 1961 - 1963)
Just Kidding …

Efekt chwytania jedynek (*ones catching*)

© Jerzy Tyszer 2017

Zatrzask typu T

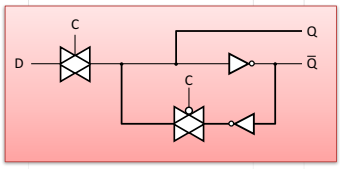


T	Q_t	Q_{t+1}
0	0	0
0	1	1
1	0	1
1	1	0

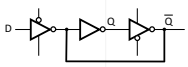
- Zwarcie wejść zatrzasku JK
- Aktywne wejście odwraca stan wyjścia

© Jerzy Tyszer 2017

Zatrzask D w CMOS



C = 0: pamiętanie
C = 1: zapis

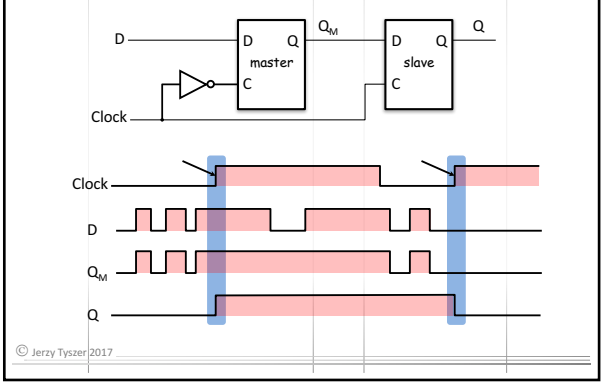


- Łącznie 10 tranzystorów – 2 inwertery (4), 2 bramki transmisyjne (4), inwerter sygnału C (2)
- Alternatywnie: 2 inwertery trójstanowe (8) i jeden inwerter konwencjonalny (2)
- Implementacja na bramkach logicznych – 18 tranzystorów

© Jerzy Tyszer 2017

Wyzwalanie zboczem

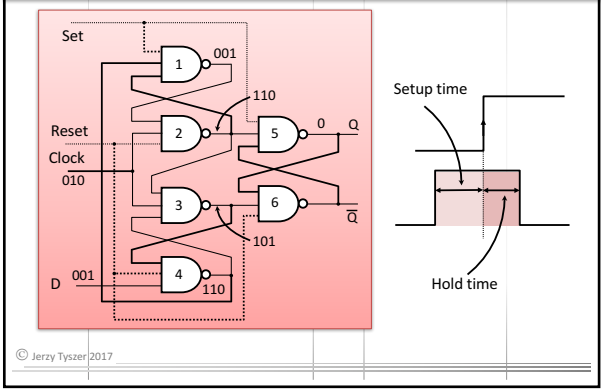
układy sekwencyjne 7



© Jerzy Tyszer 2017

Przerzutnik typu D

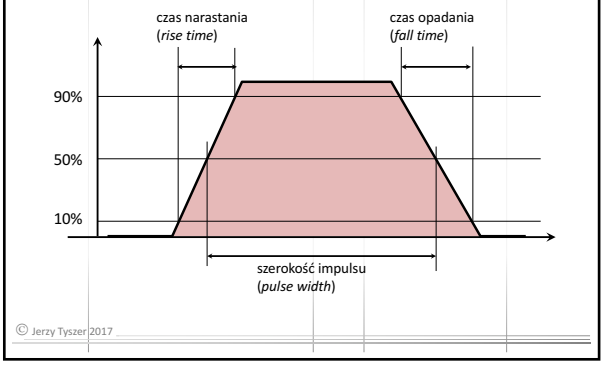
układy sekwencyjne 8



© Jerzy Tyszer 2017

Parametry czasowe

układy sekwencyjne 9



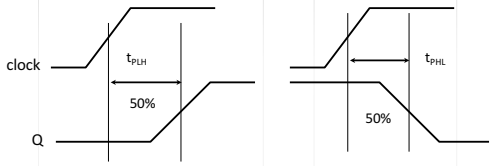
© Jerzy Tyszer 2017

Czas propagacji

- Odstęp czasu między zboczem impulsu wejściowego i wywołanym przezeń zboczem impulsu wyjściowego
- Parametr t_{PLH} (*rise delay*): przy przejściu napięcia wyjściowego od poziomu niskiego do poziomu wysokiego
- Parametr t_{PHL} (*fall delay*): przy przejściu od poziomu wysokiego do niskiego
- Akumulacja opóźnień
- W katalogach dostępne wartości nominalne, maksymalne i minimalne

© Jerzy Tyszer 2017

Czas propagacji

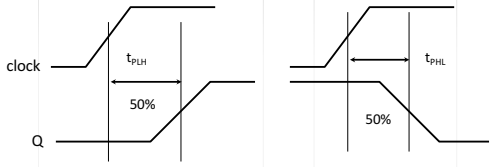


Maksymalna częstotliwość pracy układu = T^{-1}

T = minimalny *setup time* + minimalny *hold time* +
+ maksymalny czas propagacji przez bramki +
+ maksymalny czas propagacji przez przerzutniki

© Jerzy Tyszer 2017

Czas propagacji



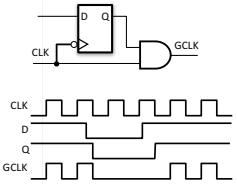
Maksymalna częstotliwość pracy układu = T^{-1}

T = minimalny *setup time* + minimalny *hold time* +
+ maksymalny czas propagacji przez bramki +
+ maksymalny czas propagacji przez przerzutniki

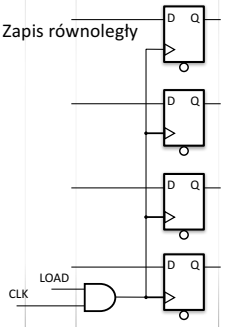
© Jerzy Tyszer 2017

Proste rejestry

Bramkowanie zegara

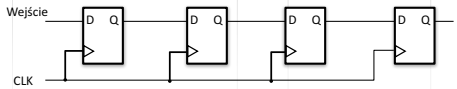


Zapis równoległy

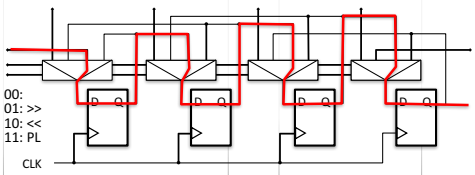


© Jerzy Tyszer 2017

Rejestr przesuwający



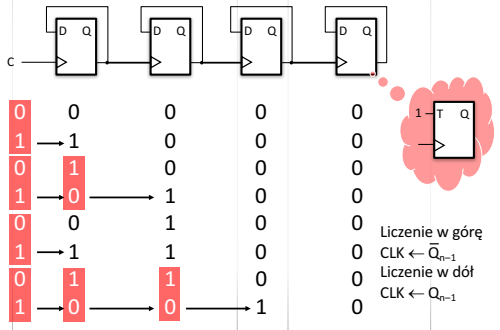
Działa prawidłowo, dopóki czas propagacji przerzutnika jest dłuższy od czasu propagacji zegara do poszczególnych segmentów rejestru



© Jerzy Tyszer 2017

Licznik asynchroniczny

układy sekwencyjne 14

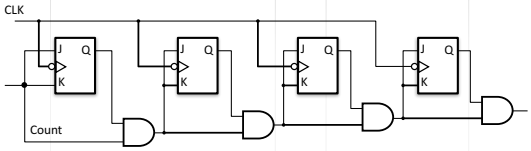


Liczenie w górę
CLK ← $\overline{Q}_{n-1}$
Liczenie w dół
CLK ← Q_{n-1}

© Jerzy Tyszer 2017

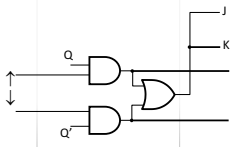
Licznik synchroniczny

układy sekwencyjne 15



Wejścia JK = 11 danego przerzutnika, jeżeli wszystkie poprzedzające bity są równe 1 oraz pojawia się sygnał Count

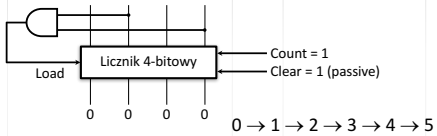
Count =1: dzielnik sygnału CLK



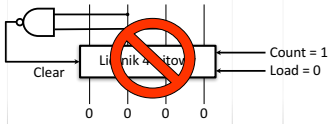
© Jerzy Tyszer 2017

Liczniki modulo N

układy sekwencyjne 16



$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$

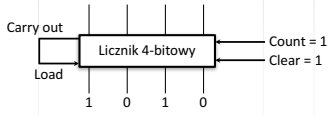


Impulsy

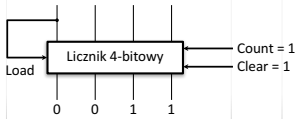
0 1 0 1
0 1 1 0
0 0 0 0

© Jerzy Tyszer 2017

Liczniki modulo 6



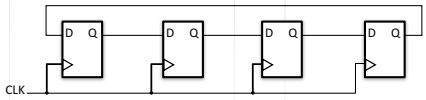
$10 \rightarrow 11 \rightarrow 12 \rightarrow 13 \rightarrow 14 \rightarrow 15$



$3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8$

© Jerzy Tyszer 2017

Licznik pierścieniowy

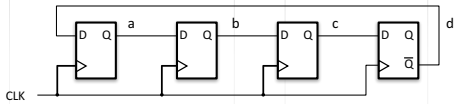


- W układzie krąży pojedyncza jedynka
- Kod wyjściowy: 1 z n
- Nie trzeba dekodować stanu licznika
- Długość cyklu (n) równa liczbie przerzutników
- Liczba stanów niewykorzystanych: $2^n - n$

© Jerzy Tyszer 2017

Licznik Johnsona

Twisted-ring counter



	a	b	c	d	dekoder
1	0	0	0	0	a'd'
2	1	0	0	0	ab'
3	1	1	0	0	bc'
4	1	1	1	0	cd'
5	1	1	1	1	ad
6	0	1	1	1	a'b
7	0	0	1	1	b'c
8	0	0	0	1	c'd

1	0	1	0	1
2	0	0	1	0
3	1	0	0	1
4	0	1	0	0
5	1	0	1	0
6	1	1	0	1
7	0	1	1	0
8	1	0	1	1

cykl niepoprawny

Liczba stanów: 2n

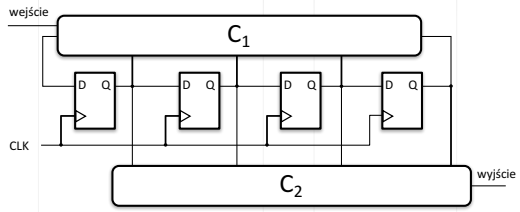
Układ nie jest bezpieczny

Korekcja:
c ← (a + c)b

© Jerzy Tyszer 2017

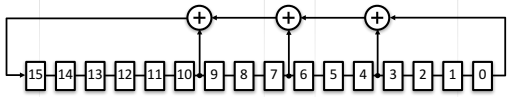
Rejestr przesuwający cd.

Rejestr przesuwający ze sprzężeniem zwrotnym



© Jerzy Tyszer 2017

Rejestry liniowe (LFSR)



$h(x) = x^{16} + x^{10} + x^7 + x^4 + 1 \rightarrow$ 16 10 7 4 0

- Dla każdej długości n rejestru liniowego istnieje pewna liczba konfiguracji sprzężenia zwrotnego, które zapewniają wygenerowanie $2^n - 1$ stanów
- Każdy wielomian pierwotny generuje inną sekwencję maksymalną (m-sekwencja)
- Konfiguracja Fibonacciego

© Jerzy Tyszer 2017

Wielomiany pierwotne

Generator sprzętowy liczb pseudolosowych

$h(x) = x^4 + x + 1$

4 1 0

Liczba wielomianów pierwotnych:

$\lambda(n) = \Phi(2^n - 1) / n$

$\Phi(k)$ – funkcja Eulera - liczba liczb względnie pierwszych z k i mniejszych od k

0 0 0 1	1	16 5 3 2 0	24 7 6 4 3 1 0
1 0 0 0	8	16 5 4 3 2 1 0	25 3 0
0 1 0 0	4	17 3 0	25 3 2 1 0
0 0 1 0	2	17 3 2 1 0	26 6 2 1 0
1 0 0 1	9	18 7 0	26 7 6 5 4 2 0
1 1 0 0	12	18 5 2 1 0	27 5 2 1 0
0 1 1 0	6	19 5 2 1 0	27 7 6 5 2 1 0
1 0 1 1	11	19 5 4 3 2 1 0	28 3 0
0 1 0 1	5	20 3 0	28 6 4 1 0
1 0 1 0	10	20 6 4 1 0	29 2 0
1 1 0 1	13	21 2 0	29 4 2 1 0
1 1 1 0	14	21 5 2 1 0	30 6 4 1 0
1 1 1 1	15	22 1 0	30 7 5 3 2 1 0
0 1 1 1	7	22 5 4 3 0	31 3 0
0 0 1 1	3	23 5 0	31 3 2 1 0
0 0 0 1	1	23 5 3 1 0	32 7 6 2 0
		24 4 3 1 0	32 7 5 3 2 1 0

© Jerzy Tyszer 2017

Wielomiany pierwotne

układy sekwencyjne 23

Niech $N = 2^n - 1$ (liczba Mersenne'a). Wielomian pierwotny *w* stopnia n: nierozkładalny, dzieli $x^N + 1$, nie dzieli $x^d + 1$, $d < N$.

Wystarczy test dla każdego czynnika pierwszego p liczby $2^n - 1$. Sprawdzamy podzielność wielomianu $x^{N/p} + 1$ przez *w*.

n = 4

$N = 2^4 - 1 = 15$

$x^{15} + 1 \,/\, x^4 + x + 1 = x^{11} + x^8 + x^7 + x^5 + x^3 + x^2 + x + 1$, R = 0

d = 5

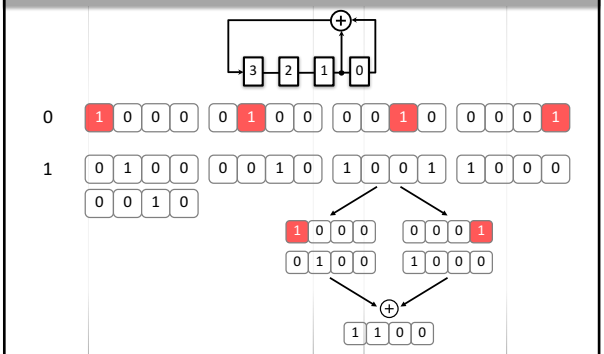
$x^5 + 1 \,/\, x^4 + x + 1 = x$, $R = x^2 + x + 1$

$x^{15} + 1 \,/\, x^4 + x^2 + 1 = x^{11} + x^9 + x^5 + x^3$, $R = x^3 + 1$

© Jerzy Tyszer 2017

Szybka symulacja

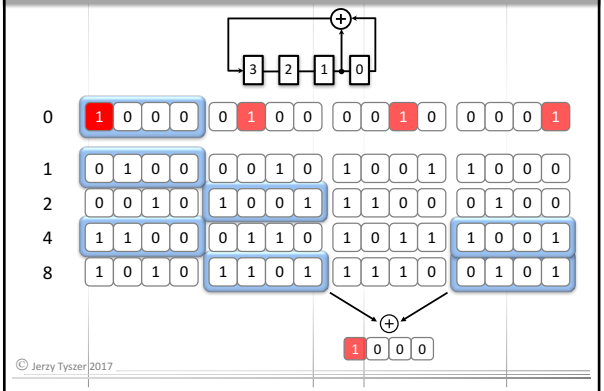
układy sekwencyjne 24



© Jerzy Tyszer 2017

Szybka symulacja – O(n^2)

układy sekwencyjne 25

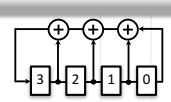


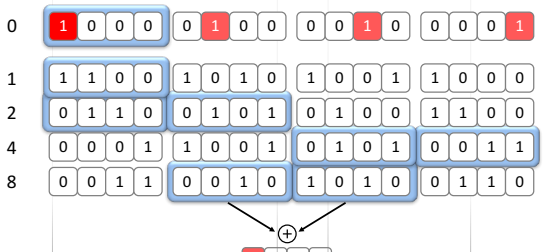
© Jerzy Tyszer 2017

Test negatywny

układy sekwencyjne 26

$h(x) = x^4 + x^3 + x^2 + x + 1$



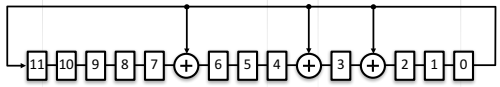


1 0 0 0 Ten sam wynik dla 5 cykli !

© Jerzy Tyszer 2017

Rejestr liniowy Galois

układy sekwencyjne 27



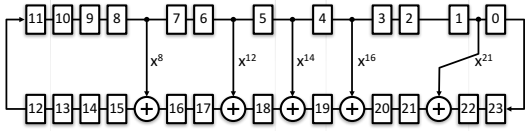
$h(x) = x^{12} + x^7 + x^4 + x^3 + 1 \rightarrow 12\ 7\ 4\ 3\ 0$

- Mniejsze opóźnienie w torze sprzężenia zwrotnego (pojedyncza bramka XOR)
- Większy rozpływ sygnału na wyjściu ostatniego segmentu
- Dla wielomianu pierwotnego generuje sekwencję maksymalną

© Jerzy Tyszer 2017

Generator pierścieniowy

układy sekwencyjne 28

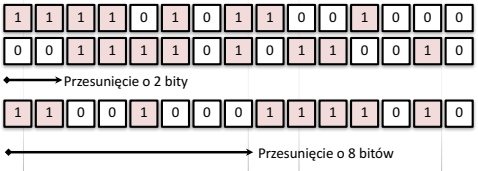


- Opóźnienie – pojedyncza bramka XOR
- Rozpływ wewnętrzny – 2
- Krótkie linie sprzężeń zwrotnych
- Układ modularny z możliwością reorganizacji sprzężenia
- Generuje sekwencję maksymalną

© Jerzy Tyszer 2017

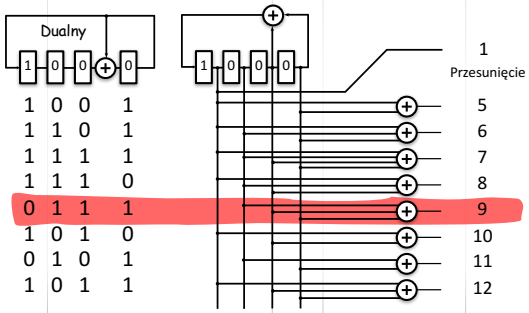
Przesuwanie fazy

- Suma dwóch sekwencji maksymalnych jest sekwencją maksymalną
- Właściwy dobór sekwencji wyjściowych umożliwia otrzymanie sekwencji przesuniętej o wymaganą liczbę bitów



© Jerzy Tyszer 2017

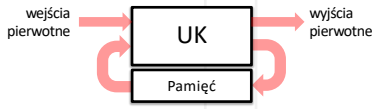
Synteza przesuwnika



© Jerzy Tyszer 2017

Klasyfikacja

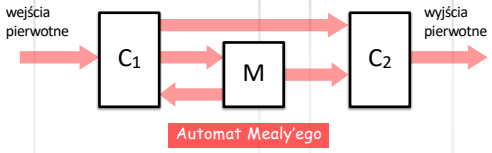
- Model Huffmana



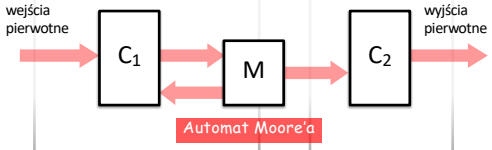
- Klasy układów sekwencyjnych
  - układy asynchroniczne
  - układy synchroniczne

© Jerzy Tyszer 2017

Dwie konwencje



Automat Mealy'ego



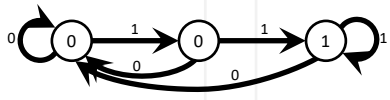
Automat Moore'a

© Jerzy Tyszer 2017

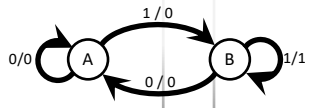
Prosty automat

Zmień pierwszą jedynkę w każdym ciągu jedynek na zero.
Pozostałe bity bez zmian: 001110110 → 000110010

Model Moore'a



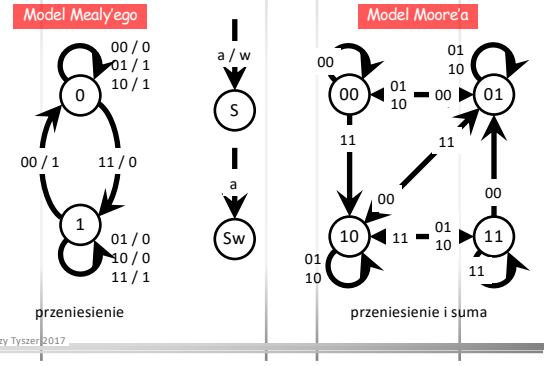
Model Mealy'ego



© Jerzy Tyszer 2017

Sumator sekwencyjny

Model Mealy'ego



przeniesienie

Model Moore'a

przeniesienie i suma

© Jerzy Tyszer 2017

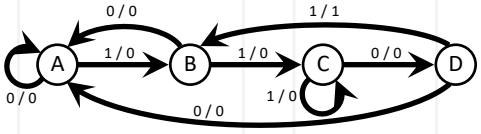
Synteza automatów

- Opis problemu
- Graf przejść i tablica stanów (*state table*)
- Redukcja stanów
- Kodowanie stanów
- Liczba i typ przerzutników
- Tablica przejść i wyjść (*transition table*)
- Funkcje wzbudzeń przerzutników i funkcje wyjść
- Realizacja układu (diagram logiczny)

© Jerzy Tyszer 2017

Synteza właściwa

Automat Mealy'ego przechodzący na wyjściu w stan 1 każdorazowo po rozpoznaniu w ciągu wejściowym sekwencji 1101

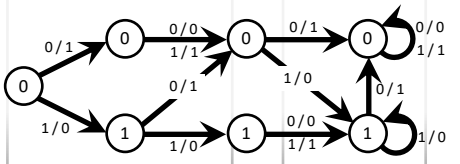


stan poprzedni	x = 0	x = 1
A	A, 0	B, 0
B	A, 0	C, 0
C	D, 0	C, 0
D	A, 0	B, 1

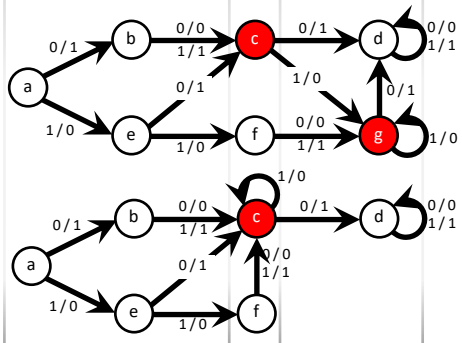
© Jerzy Tyszer 2017

Dwa stany są równoważne, jeżeli dla każdej kombinacji wejściowej dają taką samą wartość wyjścia oraz przenoszą automat do tego samego stanu lub stanów równoważnych

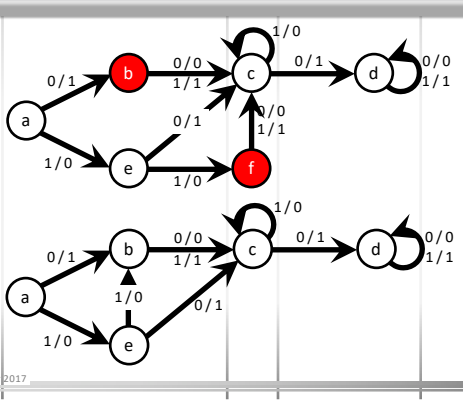
Automat dodający 5 (101) do sekwencji wejściowej



© Jerzy Tyszer 2017



© Jerzy Tyszer 2017



© Jerzy Tyszer 2017

Algorytm

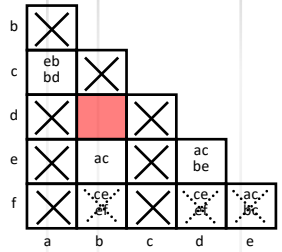
- Na podstawie tabeli przejść zbuduj tablicę trójkątną, której elementy odpowiadają kolejnym parom stanów automatu
- Dla każdego elementu tablicy i każdej kombinacji wejściowej sprawdź odpowiedź i stany następne; jeżeli odpowiedzi są różne, skreśl tę parę; jeżeli stany następne x, y są różne od obecnych i $x \neq y$, zapisz parę (x, y) w tabeli
- Sprawdź wszystkie elementy niepuste; jeżeli pozycja (x, y) zawiera zapis (u, v) i para (u, v) jest skreślona, skreśl parę (x, y)
- Powtórz ostatni krok, aż nie da się niczego skreślić; pozycje nieskreślone są odpowiednikami stanów równoważnych

© Jerzy Tyszer 2017

Stany równoważne

	00	01	10
a	e,1	c,0	b,1
b	c,0	f,1	e,1
c	b,1	a,0	d,1
d	c,0	f,1	e,1
e	a,0	f,1	b,1
f	c,0	e,1	c,1

	00	01	10
s_1	s_2,1	s_1,0	s_2,1
s_2	s_1,0	s_3,1	s_2,1
s_3	s_1,0	s_2,1	s_1,1



$a \equiv c,\ b \equiv d,\ b \equiv e,\ d \equiv e \rightarrow b \equiv d \equiv e$

$s_1 = \{a, c\},\ s_2 = \{b, d, e\},\ s_3 = \{f\}$

© Jerzy Tyszer 2017

Automaty niezupełne

	0	1
a	b,–	e,0
b	b,1	e,–
c	f,0	c,0
d	b,1	a,1
e	d,0	c,–
f	d,–	c,1

b,1; e,0; c,1; d,0

$a \equiv b,\ e \equiv f$

	0	1
a’	a’,1	d’,0
b’	d’,0	b’,0
c’	a’,1	a’,1
d’	c’,0	b’,1

b,0; e,1; c,0; d,1

$a \equiv c \equiv e$

$b \equiv d \equiv f$

	0	1
a’	b’,0	a’,0
b’	b’,1	a’,1

Oprócz nieokreślonych wyjść, w niektórych stanach mogą być także pominięte pewne kombinacje wejściowe

© Jerzy Tyszer 2017

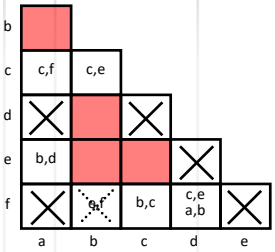
Zgodność stanów

- Dwa stany są zgodne, jeżeli dla każdej kombinacji wejściowej
  - dają taką samą wartość wyjścia, o ile jest ono określone
  - stany następne są stanami zgodnymi, o ile zostały określone; w szczególności zgodne są stany, których następniki x oraz y są niesprzeczne, tj. x = y lub x = – lub y = –
- Stany zminimalizowanego automatu niezupełnego muszą spełniać 2 warunki
  - zawierają wszystkie stany automatu wyjściowego (pokrycie)
  - dla wszystkich stanów automatu wyjściowego zawartych w pojedynczym stanie automatu zminimalizowanego i dla każdego wejścia stany następne automatu wyjściowego muszą należeć do pojedynczego stanu automatu zminimalizowanego (domknięcie)

© Jerzy Tyszer 2017

Stany zgodne

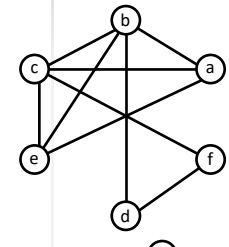
	00	01	11	10
a	b,1	f,1	d,0	–
b	–	–	–	e,1
c	b,1	c,1	–	c,1
d	e,1	c,0	a,1	–
e	–	–	b,0	–
f	c,1	–	b,1	f,1

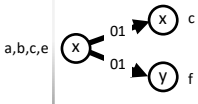


Procedura podobna do identyfikacji stanów równoważnych
Uwaga: relacja zgodności nie jest przechodnia

© Jerzy Tyszer 2017

Graf zgodności stanów





Krawędzie łączą pary węzłów, które nie zostały wykreślone w tablicy trójkątnej

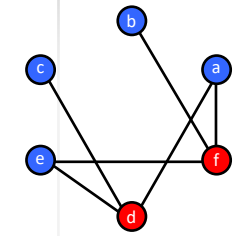
Próba ustalenia klas zgodności (identyfikacja klik w grafie):
(a,b,c,e) → x (d,f) → y

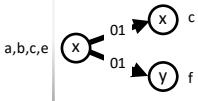
	00	01	11	10
x	x,1	?,1		
y	x,1	x,0	x,1	y,1

Źle wybrane klasy zgodności

© Jerzy Tyszer 2017

Graf niezgodności stanów





Krawędzie łączą pary węzłów, które zostały wykreślone w tablicy trójkątnej

Próba ustalenia klas zgodności (kolorowanie grafu):
(a,b,c,e) → x (d,f) → y

	00	01	11	10
x	x,1	?,1		
y	x,1	x,0	x,1	y,1

© Jerzy Tyszer 2017

Wybór poprawny

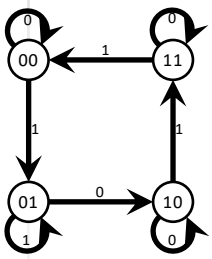
- Pary stanów (np. c,f) tworzące warunki zgodności stanów (tu: a,c) z danej grupy muszą także należeć do jednej grupy
- Należy także wybrać możliwie najmniej liczny zbiór klas zgodności (minimalna liczba klas zgodności = liczba chromatyczna grafu niezgodności)

(a,b) → x
(c,e) → y
(d,f) → z

	00	01	11	10
x	x,1	z,1	z,0	y,1
y	x,1	y,1	x,0	y,1
z	y,1	y,0	x,1	z,1

© Jerzy Tyszer 2017

Funkcje wzbudzeń



Wyjścia bezpośrednio z przerzutników JK

stan⁻ a b	we	stan⁺ a b	J_a	K_a	J_b	K_b
0 0	0	0 0	0	x	0	x
0 0	1	0 1	0	x	1	x
0 1	0	1 0	1	x	x	1
0 1	1	0 1	0	x	x	0
1 0	0	1 0	x	0	0	x
1 0	1	1 1	x	0	1	x
1 1	0	1 1	x	0	x	0
1 1	1	0 0	x	1	x	1

Q_t	Q_{t+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

© Jerzy Tyszer 2017

Funkcje wzbudzeń

we \ ab	00	01	11	10
0	0	1	x	x
1	0	0	x	x

$J_a = b\,\overline{we}$

we \ ab	00	01	11	10
0	x	x	0	0
1	x	x	1	0

$K_a = b\ we$

we \ ab	00	01	11	10
0	0	x	x	0
1	1	x	x	1

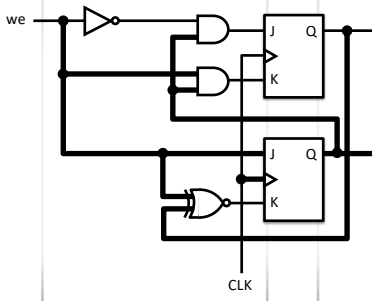
$J_b = we$

we \ ab	00	01	11	10
0	x	1	0	x
1	x	0	1	x

$K_b = \bar{a}\,\overline{we} + a\ we = \overline{(a \oplus we)}$

© Jerzy Tyszer 2017

Realizacja automatu



Q_t	Q_{t+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

Q_t	Q_{t+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

© Jerzy Tyszer 2017

Kodowanie stanów

- Różne sposoby kodowania mogą prowadzić do różnych realizacji automatu
- Wybierz kodowanie dające najprostsze funkcje wzbudzeń

	0	1
a	a,0	b,0
b	a,0	c,0
c	d,0	c,0
d	a,0	b,1

	Kodowanie (1)	(2)
a	a = 00	a = 00
b	b = 01	b = 11
c	c = 10	c = 01
d	d = 11	d = 10

$D_1 = Q_1\,\overline{Q}_2 + x\,\overline{Q}_1\,Q_2$

$D_2 = \bar{x}\,Q_1\,\overline{Q}_2 + x\,Q_1\,Q_2 + x\,\overline{Q}_1\,\overline{Q}_2$

$y = x\,Q_1\,Q_2$

$D_1 = \bar{x}\,\overline{Q}_2 + x\,\overline{Q}_1\,Q_2$

$D_2 = x$

$y = x\,Q_1\,\overline{Q}_2$

© Jerzy Tyszer 2017

Kodowanie stanów

- Dla n stanów $\rightarrow s = \lceil \log_2 n \rceil$ zmiennych stanu, a zatem dostępnych jest 2^s kombinacji
- Liczba różnych odwzorowań stanów w kody

$$L = \frac{2^s!}{(2^s - n)!} \frac{1}{s!\, 2^s} = \frac{(2^s - 1)!}{(2^s - n)!\, s!}$$

n =10, s = 4, L = 75 675 600

© Jerzy Tyszer 2017

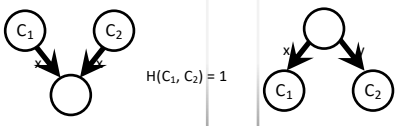
Metody kodowania stanów

- Prosta numeracja stanów
- Losowa selekcja słów kodowych
- Kodowanie „1 z n" (1-*hot encoding*) – każdy stan reprezentowany przez jeden bit (przerzutnik)
- Wykorzystanie stanu wyjść (por. automat Moore'a)
- Kodowanie za pomocą kodów „m z n" – detekcja stanów zabronionych
- Kodowanie heurystyczne
- Kodowanie dostępne w narzędziach CAD (np. programy Nova, Jedi, Kiss, Mustang)

© Jerzy Tyszer 2017

Kodowanie heurystyczne

- Przypisz podobne kody, np. odległe w sensie Hamminga o 1, do stanów „bliskich":
  - ich stany następne są takie same dla danej kombinacji wejściowej
  - są stanami następnymi tego samego stanu
  - bliskość opisana odpowiednimi wagami lub innymi regułami heurystycznymi



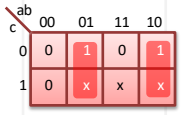
© Jerzy Tyszer 2017

Automaty bez wejść

Rejestr liniowy jest automatem przechodzącym przez kolejne stany tylko pod wpływem taktów zegara

$$0 \to 1 \to 2 \to 4 \to 6 \to 0$$

$stan^-$ a b c	$stan^+$ a b c	D_a	D_b	D_c
0 0 0	0 0 1	0	0	1
0 0 1	0 1 0	0	1	0
0 1 0	1 0 0	1	0	0
0 1 1	x x x	x	x	x
1 0 0	1 1 0	1	1	0
1 0 1	x x x	x	x	x
1 1 0	0 0 0	0	0	0
1 1 1	x x x	x	x	x

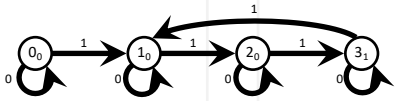


$$D_a = \bar{a}\,b + a\,\bar{b} = a \oplus b$$
$$D_b = a\,\bar{b} + c$$
$$D_c = \bar{a}\,\bar{b}\,\bar{c}$$

© Jerzy Tyszer 2017

Automat i układ iteracyjny

Zaprojektuj *n*-wejściowy *n*-wyjściowy układ iteracyjny. Wyjście *k*-tego segmentu przechodzi w stan 1, jeżeli całkowita liczba jedynek występujących na wejściach segmentów z lewej strony segmentu *k* (włączając segment *k*) jest większa od 0 oraz podzielna przez 3. Np. 010101001111 generuje 000001110010.



S_1 S_0 x_k	S_{1+} S_{0+}
0 0 0	0 0
0 0 1	0 1
0 1 0	0 1
0 1 1	1 0
1 0 0	1 0
1 0 1	1 1
1 1 0	1 1
1 1 1	0 1

$$S_{0+} = (x \oplus S_0) + x\,S_1$$
$$S_{1+} = S_1 \oplus x\,S_0$$
$$y = S_{1+}\,S_{0+}$$

© Jerzy Tyszer 2017